

SYSTEM_REVERSE_ENGINEERING.md

VSSC Infrastructure Management Portal

System Reverse Engineering Report

Version: 1.0

Last Updated: June 2026

PURPOSE OF THIS DOCUMENT

This document provides a complete reverse-engineering analysis of the existing VSSC Infrastructure Management System.

Its purpose is to help future developers and AI systems understand:

- What the system does
- How it works
- How the components interact
- What workflows exist
- How data moves through the application

This document should be considered the architectural reference for the current system.

SYSTEM OVERVIEW

What Is This System?

The application is an Infrastructure Management Portal built around Proxmox virtualization infrastructure.

The platform acts as a centralized operational dashboard for:

- Infrastructure Administrators
- Technical Managers
- Authorized Users

It provides visibility into:

- Virtual Machines

- Hypervisor Nodes
 - Clusters
 - Storage Resources
 - VM Ownership
 - Staff Allocation
 - Resource Utilization
 - Infrastructure Synchronization
-

PRIMARY BUSINESS PURPOSE

The system exists to solve four primary problems.

1. Infrastructure Visibility

Administrators need a single location to view:

- Active virtual machines
- Physical host nodes
- Storage resources
- Cluster resources

without logging directly into Proxmox.

2. Resource Ownership

Every VM must be associated with:

- Employees
- Divisions
- Entities
- Business groups

This enables accountability and auditing.

3. Capacity Monitoring

Administrators need visibility into:

- CPU allocation
- Memory allocation
- Storage allocation

- Resource growth

to avoid infrastructure exhaustion.

4. Synchronization

The database must remain synchronized with the actual Proxmox infrastructure.

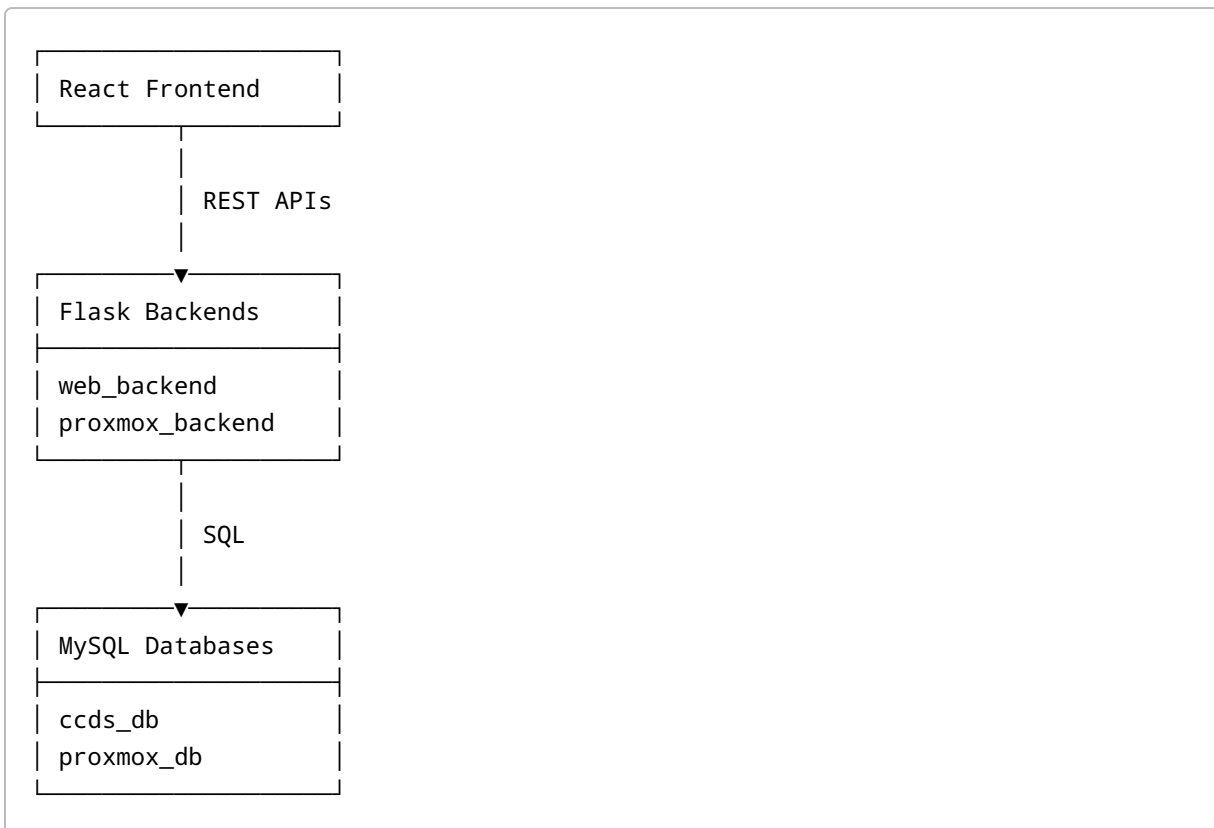
Synchronization updates:

- Clusters
- Nodes
- Storage
- Virtual Machines

from the live environment.

HIGH LEVEL ARCHITECTURE

The system follows a three-layer architecture.



DATABASE ARCHITECTURE

The platform operates on two databases.

proxmox_db

Infrastructure database.

Stores:

- Clusters
- Nodes
- Storage
- VMs
- Ownership mappings
- Staff directory

This is the operational source of truth.

ccds_db

Application database.

Stores:

- User authentication
- Roles
- VM request forms
- Requested VM ownership mappings

This is the application support database.

FRONTEND ARCHITECTURE

Original Architecture

Before redesign:

```
Frontend
├─ Pages
├─ Components
├─ API Clients
└─ Utility Hooks
```

The application relied heavily on:

- Local component state
- `useEffect`
- `Axios`

No centralized state management layer existed.

CURRENT FRONTEND ARCHITECTURE

After Stage 1 refactor:

```
src
├─ layouts
├─ pages
├─ components
├─ theme
├─ routes
├─ api
├─ hooks
└─ animations
```

Additional systems added:

- Theme Provider
 - Motion Framework
 - Role-aware navigation
 - Responsive application shell
-

BACKEND ARCHITECTURE

The platform contains two Flask services.

WEB BACKEND

Port:

5000

Purpose:

Authentication and application operations.

Responsibilities:

- Login
- Logout
- Password change
- User creation
- VM request submission
- Staff search

Database:

ccds_db

PROXMOX BACKEND

Port:

5001

Purpose:

Infrastructure management.

Responsibilities:

- Cluster synchronization
- Node synchronization
- Storage synchronization
- VM synchronization
- Ownership management
- Reporting
- Visualization data
- Monitoring metrics

Database:

proxmox_db

AUTHENTICATION ARCHITECTURE

Authentication Flow

```
graph TD; A[User Login] --> B[POST /auth/login]; B --> C[Session Created]; C --> D[Role Returned]; D --> E[Stored in Frontend];
```

The diagram illustrates the authentication flow as a vertical sequence of steps connected by downward arrows. It starts with 'User Login', followed by a POST request to '/auth/login', then 'Session Created', 'Role Returned', and finally 'Stored in Frontend'.

Session Validation

Frontend periodically calls:

GET /auth/check

to verify session validity.

Invalid sessions trigger:

- Logout
 - Storage cleanup
 - Redirect to login
-

ROLE SYSTEM

The original system contained:

- admin
- view_only

The redesign introduces:

- admin
- manager
- user

through frontend role handling.

CORE BUSINESS ENTITIES

Virtual Machine

Represents a virtualized compute resource.

Attributes include:

- Name
- UUID
- CPU
- RAM
- Disk
- OS
- Status
- GPU
- Ownership

Relationships:

- Cluster
 - Node
 - Storage
 - Users
-

Node

Represents a physical hypervisor.

Contains:

- CPU resources
 - Memory resources
 - Hypervisor information
 - Uptime
 - Connectivity status
-

Cluster

Represents a logical Proxmox environment.

Contains:

- Multiple nodes
 - Multiple VMs
 - Multiple storage resources
-

Storage

Represents attached storage resources.

Contains:

- Capacity
 - Type
 - Datastore information
 - Availability
-

Employee

Represents an individual in VSSC.

Contains:

- Staff code
- Division
- Group
- Entity

Used for ownership mapping.

SCREEN ARCHITECTURE

Login

Purpose:

Authenticate users.

Responsibilities:

- Login
 - Error handling
 - Session creation
-

Dashboard

Purpose:

Provide infrastructure overview.

Displays:

- VM statistics
 - Resource usage
 - Capacity indicators
 - Allocation summaries
-

Infrastructure

Contains:

- Clusters
- Nodes
- Storage

Purpose:

Provide infrastructure visibility.

Virtual Machines

Purpose:

Central VM inventory.

Features:

- Search
 - Filters
 - Ownership
 - Storage mapping
 - Exporting
-

Monitoring

Purpose:

Infrastructure pressure analysis.

Uses:

- CPU pressure
- Memory pressure
- IO pressure

data from Proxmox.

Analytics

Purpose:

Resource allocation analysis.

Includes:

- Entity usage
 - Division usage
 - Capacity trends
-

Reports

Purpose:

Generate exportable infrastructure reports.

Formats:

- CSV
 - PDF
 - XLSX
-

Administration

Purpose:

Administrative operations.

Includes:

- User management
 - Sync operations
 - VM requests
-

CORE WORKFLOWS

Workflow 1: Login

```
graph TD; User --> Login_Screen[Login Screen]; Login_Screen --> POST[/auth/login/]; POST --> Authenticated; Authenticated --> Dashboard;
```

User
|
▼
Login Screen
|
▼
POST /auth/login
|
▼
Authenticated
|
▼
Dashboard

Workflow 2: Infrastructure Synchronization

```
graph TD; Admin --> Sync_Center[Sync Center]; Sync_Center --> Cluster_Sync[Cluster Sync]; Cluster_Sync --> Node_Sync[Node Sync]; Node_Sync --> Storage_Sync[Storage Sync]; Storage_Sync --> VM_Sync[VM Sync]; VM_Sync --> Database_Updated[Database Updated]; Database_Updated --> UI_Refresh[UI Refresh];
```

Admin
|
▼
Sync Center
|
▼
Cluster Sync
|
▼
Node Sync
|
▼
Storage Sync
|
▼
VM Sync
|
▼
Database Updated
|
▼
UI Refresh

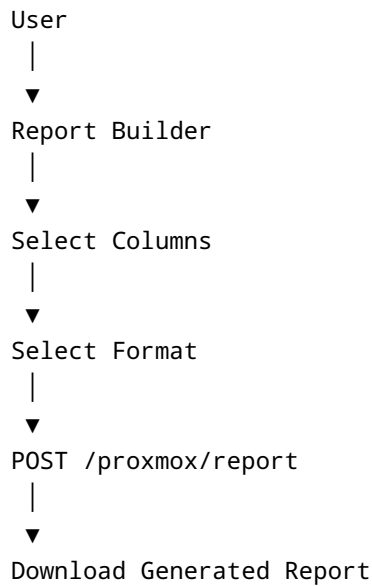
Workflow 3: VM Ownership Assignment

```
graph TD; Admin --> VM_Details[VM Details]; VM_Details --> Search_Staff[Search Staff]; Search_Staff --> Add_User[Add User]; Add_User --> Create_vm_user_relation[Create vm_user_relation]; Create_vm_user_relation --> Ownership_Updated[Ownership Updated];
```

Workflow 4: Capacity Monitoring

```
graph TD; User --> Monitoring_Screen[Monitoring Screen]; Monitoring_Screen --> Select_Nodes[Select Nodes]; Select_Nodes --> Request_Metrics[Request Metrics]; Request_Metrics --> Render_Pressure_Charts[Render Pressure Charts];
```

Workflow 5: Reporting



DATA FLOW ANALYSIS

Frontend To Backend

Uses Axios clients.

Two API layers:

webApi

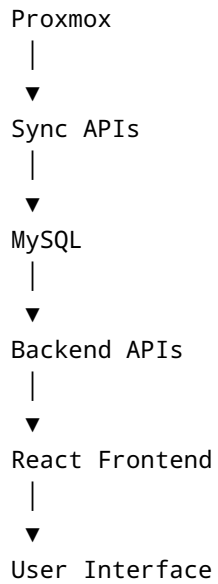
proxmoxApi

Backend To Database

Uses:

- SQLAlchemy
 - MySQL queries
 - Proxmox synchronization services
-

Data Lifecycle



CURRENT STRENGTHS

Backend

Well separated responsibilities.

Dedicated infrastructure backend.

Dedicated authentication backend.

Database

Strong relational structure.

Clear ownership model.

Good normalization.

Infrastructure Integration

Live synchronization support.

Monitoring support.

Reporting support.

ORIGINAL WEAKNESSES

Before redesign:

- Weak dashboard
 - Minimal analytics
 - Outdated UI
 - Poor navigation
 - Limited role separation
 - Weak visual hierarchy
-

STAGE 1 IMPROVEMENTS

Completed:

- New shell
 - New navigation
 - Theme architecture
 - RBAC foundation
 - Analytics foundation
 - Infrastructure integrations
-

KNOWN RISKS

Sync Operations

Must remain functional.

Critical workflow.

Any frontend modification must preserve sync behavior.

Ownership Filtering

User role visibility depends entirely on:

vm_user_relation

Incorrect filtering may expose infrastructure data.

Reporting

Exports depend on backend formatting.

Frontend should remain format-agnostic.

FUTURE ARCHITECTURE GOALS

Dashboard

More operational visibility.

VM Management

Modern drawer-based workflows.

Monitoring

Dedicated monitoring center.

Analytics

Expanded infrastructure intelligence.

User Experience

Dedicated user dashboards.

Role-specific experiences.

DOCUMENT USAGE

This document should be read:

1. Before modifying architecture.
2. Before redesigning workflows.
3. Before introducing new analytics.
4. Before implementing new role behavior.
5. Before major frontend refactors.

It serves as the architectural reference for the entire VSSC Infrastructure Management Portal.